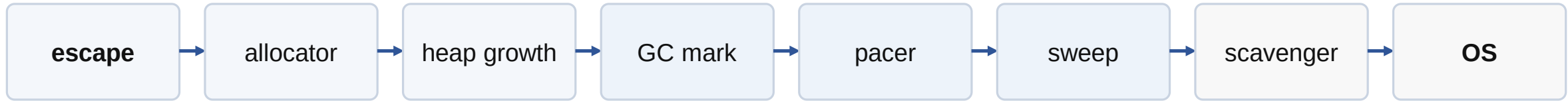


Доп. лекция 3. Go GC internals и memory allocator

mcache → mspan → mcentral → mheap · size classes · Green Tea · write barrier · pacer · assists · sweep · scavenger

Глубокий runtime-блок к лекции 3. Версионная рамка: Go 1.27.x.



Главный вопрос: что реально происходит после решения compiler «нужна heap allocation»?

Allocator, GC, scheduler и virtual memory ОС здесь рассматриваются как одна система.

Устойчивая инженерная модель

- per-P fast path
- size-segregated allocation
- tracing concurrent GC
- write barriers
- GC pacing
- mutator assists
- sweep + scavenging

Current implementation details

- ``mcache``, ``mspan``, ``mcentral``, ``mheap``
- конкретные bitmaps и поля
- Green Tea internal queues
- точные pacer constants
- точная madvise policy
- size-specialized routines Go 1.27

Runtime source ≠ спецификация языка.

Почему «runtime выделил память» — недостаточное объяснение

```
func newUser() *User {  
    u := User{ID: 42}  
    return &u  
}
```

compiler: u escapes

Кто найдёт свободную память?

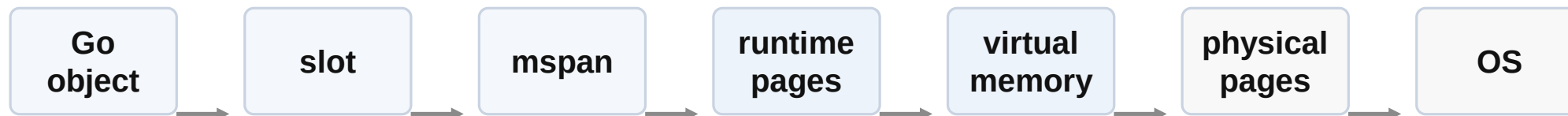
Нужен ли lock?

Будет ли syscall?

Кто потом вернёт memory?

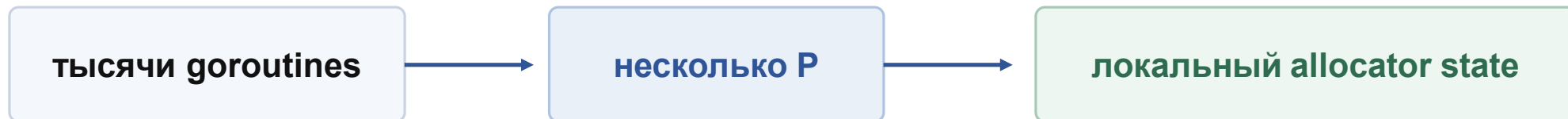
Heap allocation — вход в runtime pipeline, а не один вызов «дай память».

Memory живёт на разных масштабах



ОС не знает, что существует `User`. Для ядра это страницы virtual memory процесса.

Почему allocator иерархический

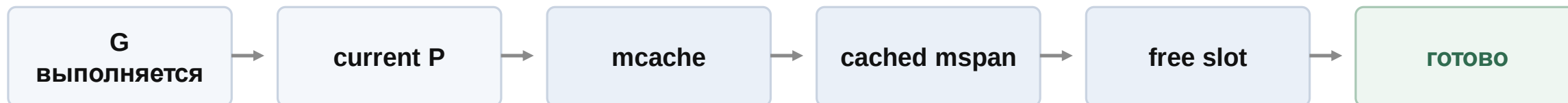


**common path: без глобального
allocator lock**

**rare slow path: shared structures →
page allocator → OS**

local fast path + shared slow path

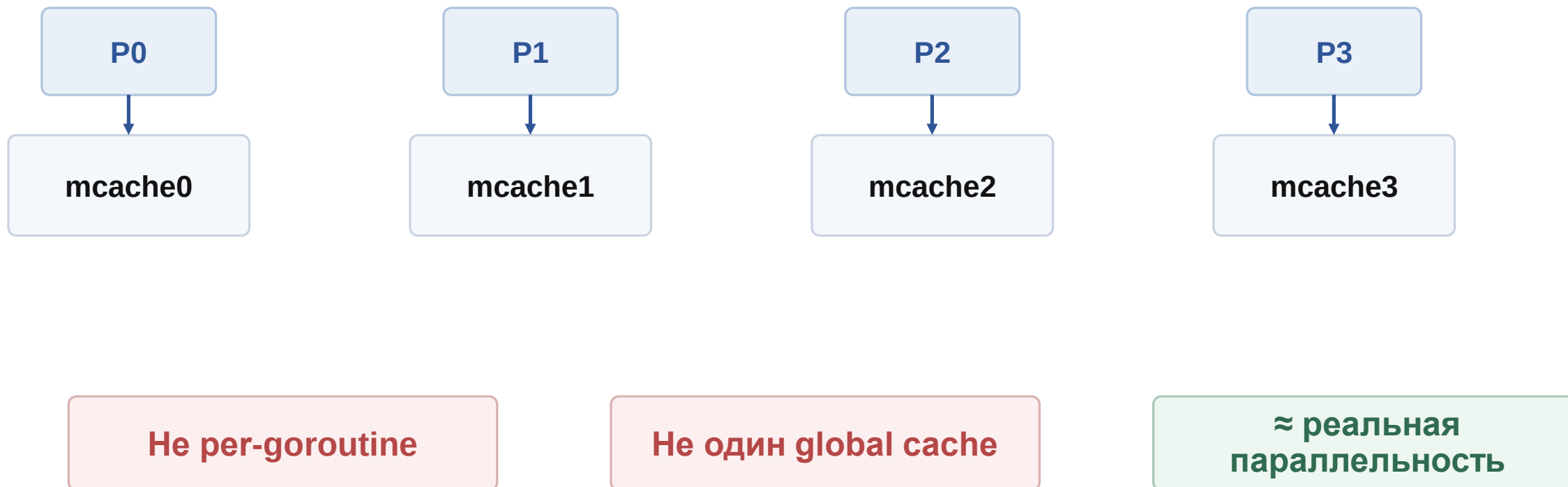
Fast path маленькой allocation



Нет mcentral. Нет mheap. Нет mmap. Нет глобального lock.

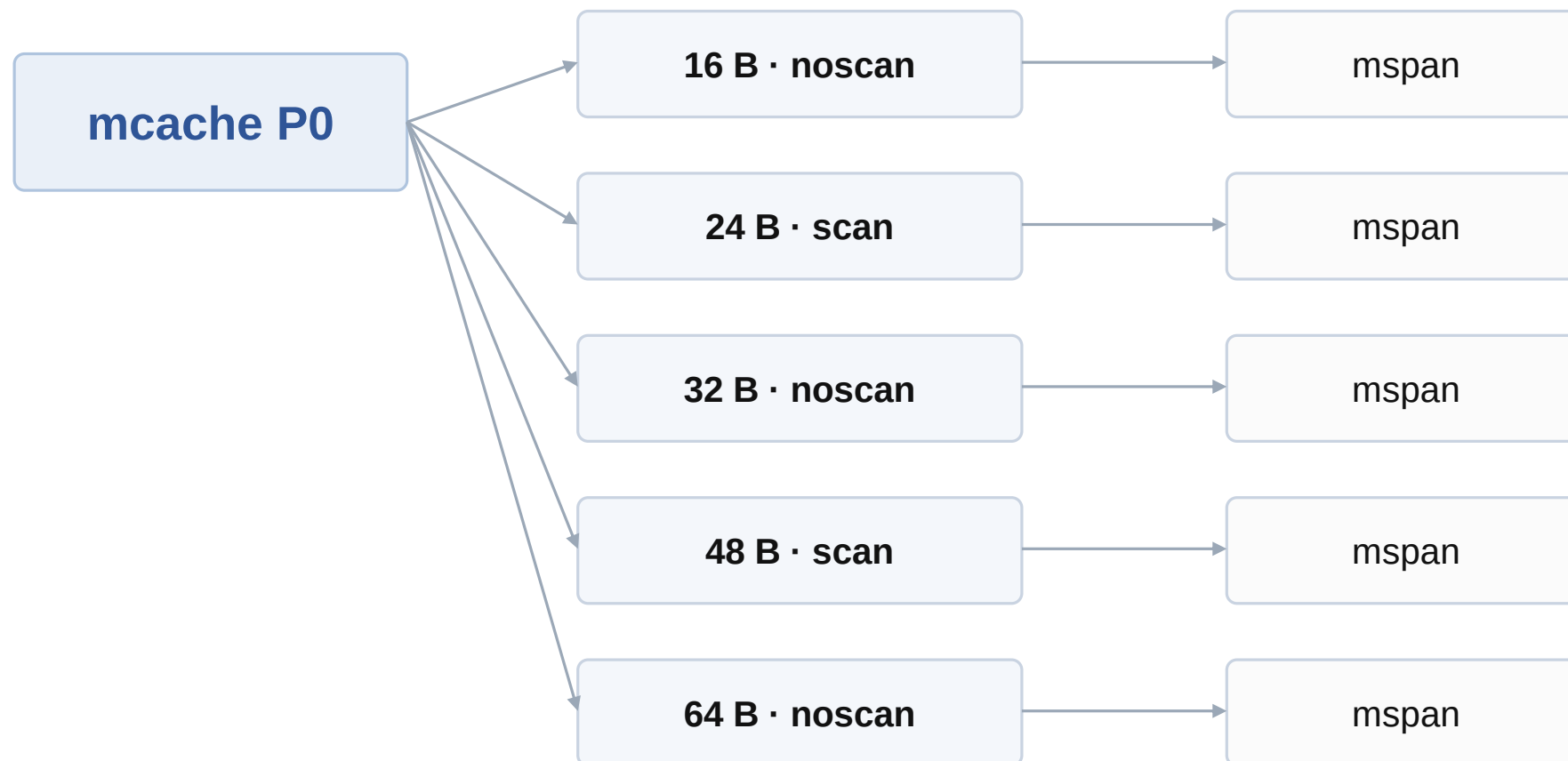
Это и есть причина, почему маленькая allocation может быть очень дешёвой прямо сейчас.

Почему allocator cache принадлежит P



P хранит и scheduler-local state, и allocator-local state: один и тот же runtime pattern локальности.

mcache: не кусок памяти, а набор готовых spans



size + pointer layout → spanClass → cached span

Size classes: платим немного memory за простой fast path

```
requested = 37 B  
size class = 48 B  
slot = 48 B  
  
internal waste = 11 B
```

Больше size classes



меньше waste
больше complexity

Меньше size classes



проще allocator
больше fragmentation

Current small allocator: classes до 32 KiB.

Конкретная таблица classes — implementation detail текущего Go.

mspan: центральная единица allocator



mspan в цифрах: slots, allocBits, freeindex

```
runtime page = 8192 B  
slot size    = 64 B
```

```
8192 / 64 = 128 slots
```

slot: 0 1 2 3 4 5 6 7

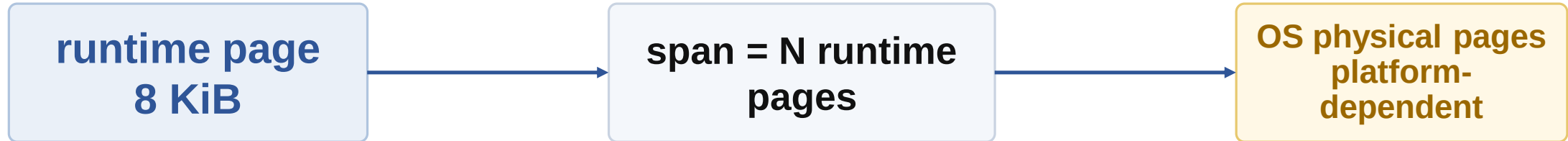
allocBits: 1 1 1 1 0 0 1 0



freeindex примерно отсюда

Allocator ищет free slot по битмап — не делает syscall на каждый object.

Runtime page $\neq$ physical page OS



Language/runtime abstraction начинает протекать в VM subsystem ОС.

8 KiB — current runtime detail, не language guarantee.

scan / noscan: одинаковые bytes, разная GC цена

Pointer-free object

```
type A struct {  
    X uint64  
    Y uint64  
}
```

noscan

Pointer-containing object

```
type B struct {  
    X *User  
    Y uint64  
}
```

scan

spanClass = size class + noscan bit

100 MB `[]byte` и 100 MB `[]*Node` — не одно и то же

`[]byte` · 100 MB

`byte byte byte byte ...`

GC: внутри нет heap pointers

`[]*Node` · 100 MB

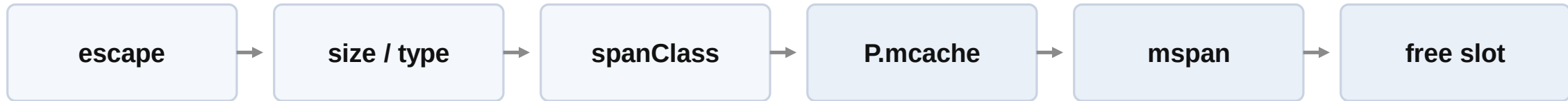
`ptr ptr ptr ptr ...`

GC: сканировать references
и продолжать graph traversal

same heap bytes ≠ same GC work

Практическая метрика: `/gc/scan/heap:bytes`.

Полный small-allocation path



slot есть → allocate

slot нет → mcentral → mheap → page allocator

OS появляется только если runtime действительно нужен новый memory region.

Почему zeroing нужен

```
старый slot:  
DE AD BE EF ...  
  
новый Go value:  
0 0 0 0 ...
```

Что оптимизирует Go 1.27

- compiler часто уже знает размер
- знает scan/noscan
- для части маленьких объектов <80 В использует specialized routines
- меньше generic branches и indirect work

Compiler ↔ runtime boundary тоже оптимизируется.

Порог и конкретные routines — Go 1.27 implementation detail.

Когда mspan заполнен: refill



Refill происходит редко по сравнению с allocations внутри span.

Одна дорогая операция амортизируется на десятки/сотни будущих allocations.

mcentral управляет spans, не отдельными objects

mcentral[64 B · noscan]

partial

span A

span B

span C

full

span D

span E

мсache получает
span

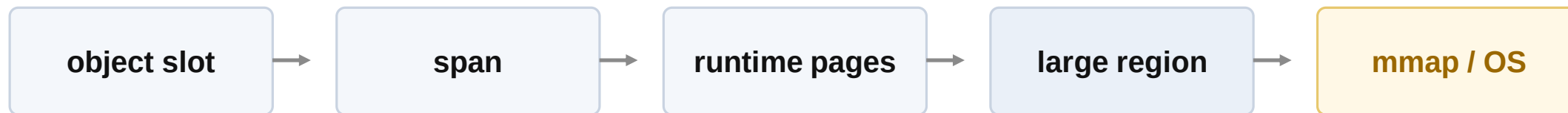
потом локально
раздаёт slots

Synchronization вынесена с hot path на
refill path.



```
page map:
used used free free free used ...
           └───┬───┘
             new span
```

**mheap думает runs of pages; objects
выдаются выше через spans.**



**64-byte allocation обычно не превращается в системный
ВЫЗОВ.**

Small, large, tiny: три разных режима

TINY

<16 B · pointer-free
несколько objects в одном
block

SMALL

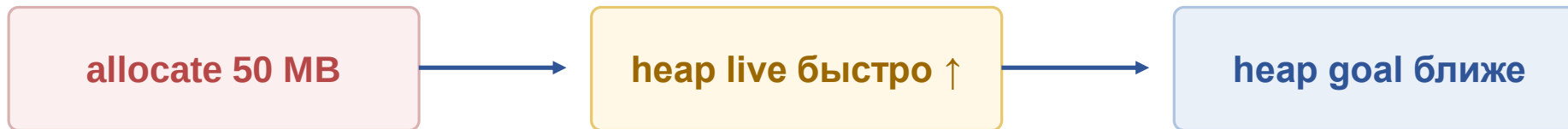
≤32 KiB
size classes +
mcache/mcentral

LARGE

>32 KiB
page-level path / own span

Один allocator — несколько paths под разные масштабы.

Почему large allocations чувствительны



fragmentation / RSS

scavenging

GC trigger timing

Large churn особенно заметен на пилообразной нагрузке.

Tiny allocator: экономим overhead на крошечных noscan objects

```
new(byte)  
new(uint16)  
new(uint32)
```

16-byte tiny block

только noscan

1 B

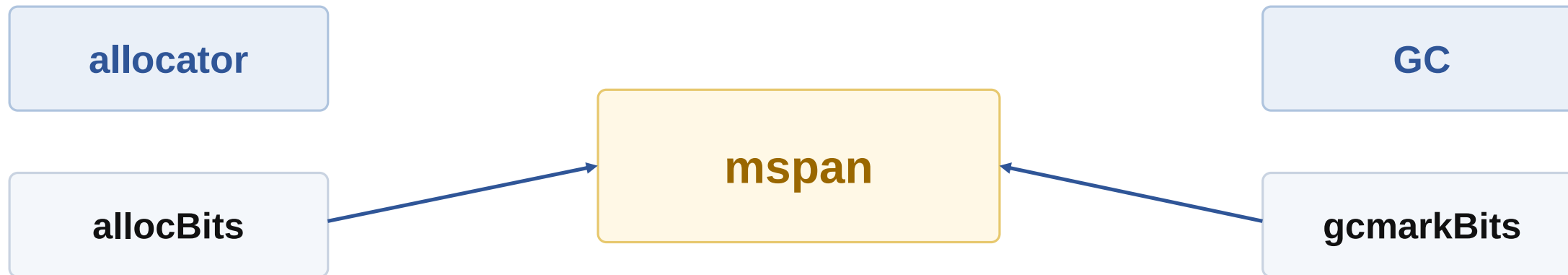
2 B

4 B

...

Цена: если один subobject жив, block может удерживаться целиком.

Где allocator встречается с GC



Object dead ≠ slot сразу reusable. Сначала collector должен доказать unreachable.

tracing

precise

concurrent

parallel

mark-and-sweep

non-generational

non-compacting

Это архитектура collector, а не набор маркетинговых ярлыков.

Механика

- tracing → идём от roots по references
- precise → runtime знает pointer layout
- concurrent → приложение работает во время mark
- parallel → несколько workers делают GC

Следствие

- non-generational → нет young/old model
- non-compacting → живые objects обычно не двигаются ради уплотнения
- fragmentation решается allocator/page management, а не compaction

«Go GC» нельзя объяснить через generational moving model другого runtime.

Concurrent graph mutation: откуда появляется write barrier

GC просканировал A

$A \rightarrow B$

mutator пишет $A \rightarrow C$

Collector и application видят меняющийся heap graph.



Нужна bookkeeping вокруг pointer writes → write barrier.

```
before:  
slot → old
```

```
write:  
slot → new
```

GC должен не потерять important references

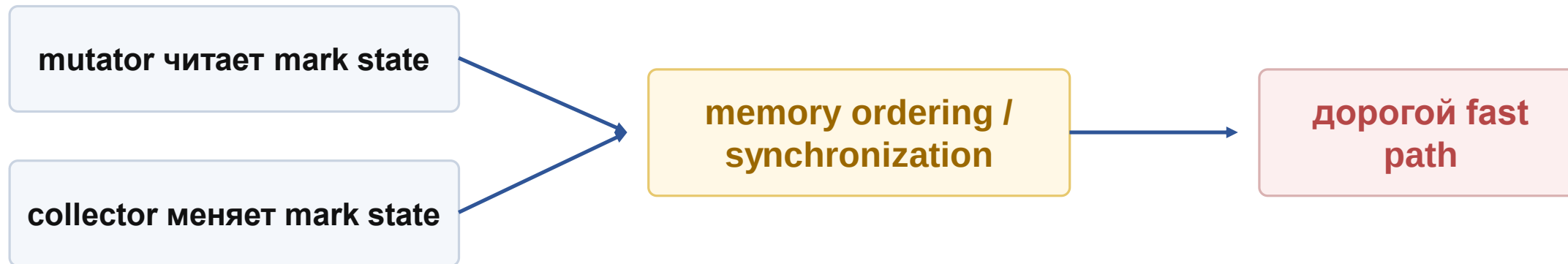
old pointer

new pointer

Barrier = дополнительная GC work вокруг store во время marking.

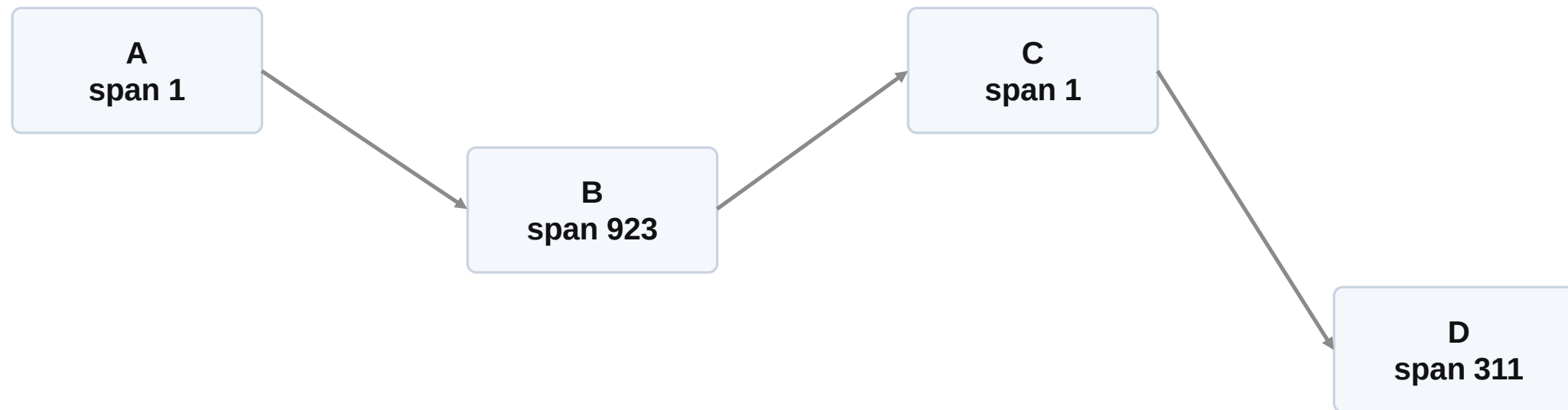
Точная комбинация Yuasa/Dijkstra invariants — detail runtime; студенту важна проблема concurrent graph mutation.

Почему нельзя просто проверять «destination уже black?»



Runtime предпочитает немного лишней shading work вместо дорогой synchronization на каждом write.

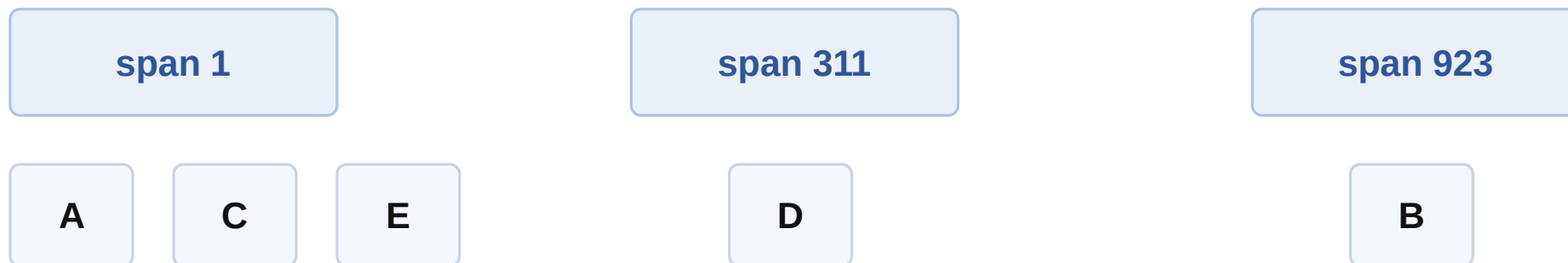
Green Tea: старая проблема — pointer chasing



pointer chasing → cache misses → DRAM latency

Marking cost часто ограничена memory locality, а не арифметикой.

Green Tea: batching work по span

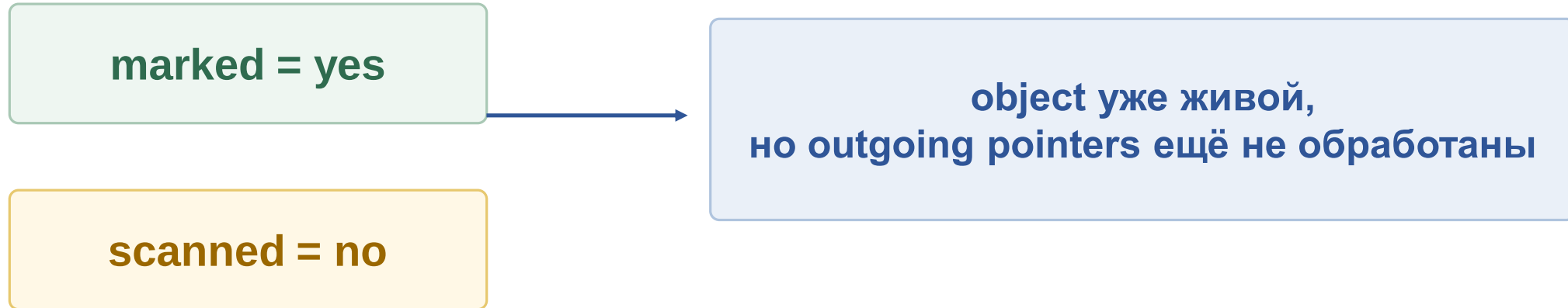


лучше reuse metadata/cache lines · меньше хаотичных переходов

Go 1.26+: Green Tea default

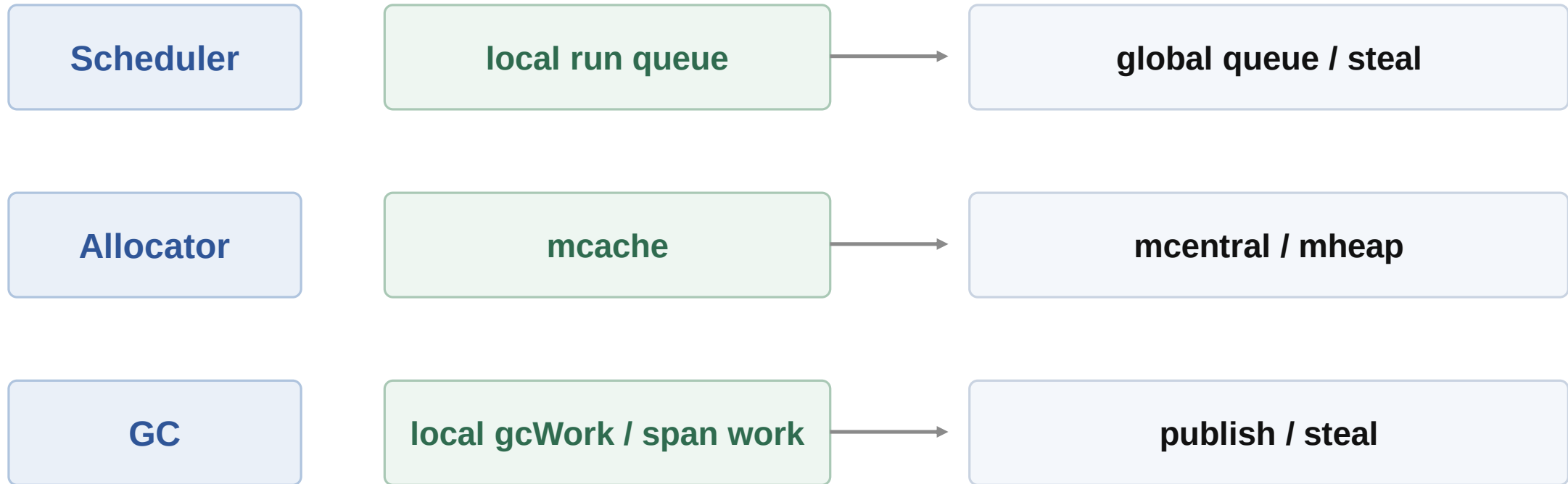
Implementation changed; programming model reachability did not.

Mark ≠ scan: Green Tea нужны два состояния



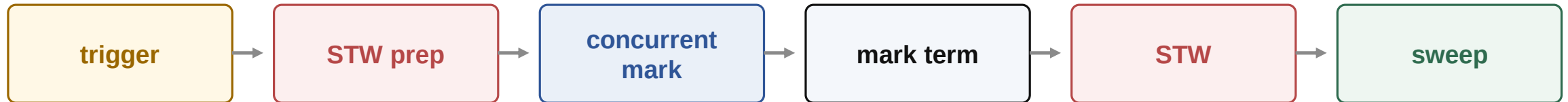
mark work можно накапливать и batch-ить по span.

Один runtime pattern повторяется везде



Локальность сначала. Глобальная координация — позже.

GC cycle как timeline



STW остаётся для коротких глобально согласованных фаз.

Кто физически делает marking

dedicated workers

fractional workers

idle workers

mutator assists

background target $\approx 25\% \times \text{GOMAXPROCS}$

Но profile GC CPU $\neq$ автоматически CPU украденный у requests: idle workers используют свободную capacity.

25% — current pacer target, а не guarantee постоянного GC CPU.

GC pacer: heap goal — финиш, trigger — старт



allocation continues while GC marks

Start on goal = уже поздно.

Pacer оценивает allocation rate, scan work, throughput, roots, memory budget.

GOGC глубже: growth budget

```
Target ≈  
Live heap +  
(Live heap + GC roots) × GOGC /  
100
```

GOGC ↓

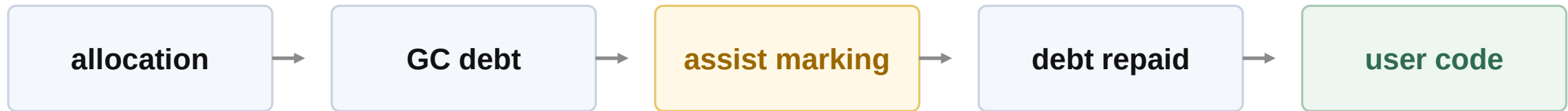
**меньше runway
чаще GC
больше CPU**

GOGC ↑

**больше runway
реже GC
больше memory**

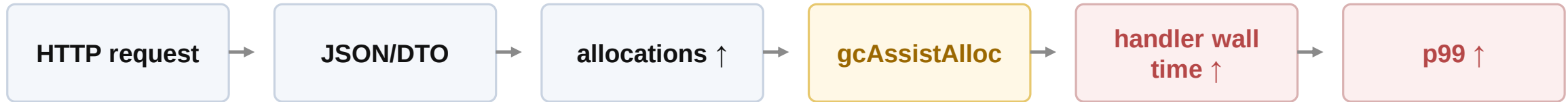
Goal — control target. Trigger рассчитывается отдельно.

Mutator assists: приложение начинает убирать за собой



Application allocates быстрее, чем collector успевает → request goroutine сама выполняет GC work.

Почему assists бьют по p99



Большой STW не нужен: latency растёт прямо внутри request goroutine.

Sweep: mark ещё не освобождает slot

allocBits 1 1 1 1

gcmarkBits 1 0 1 0



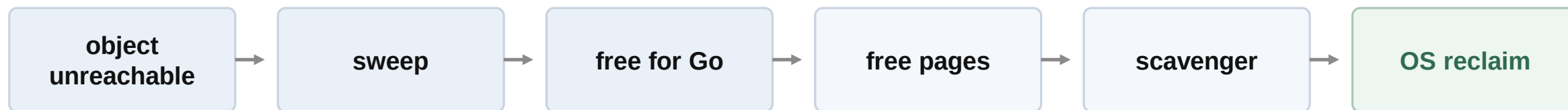
A occupied
B free
C occupied
D free

Mark: кто жив? Sweep: какие slots снова reusable?

Sweep может идти background и allocation-driven; sweepgen кодирует lifecycle span между cycles.

Sweep ≠ scavenging

RECLAIM



GC reclaim ≠ OS reclaim

Runtime может оставить free memory для быстрого reuse.

Отдать всё сразу

- RSS быстрее падает
- но потом снова page faults / kernel VM work
- дороже следующий traffic spike

Оставить всё runtime

- reuse быстрый
- меньше kernel churn
- но RSS может оставаться высоким

Scavenger ищет свободные runtime pages, physical backing которых можно release OS.

Linux: runtime memory встречается с `madvice`



MADV_FREE / MADV_DONTNEED — platform policy detail

virtual address space $\neq$ physical RSS

HeapAlloc и RSS: четыре разные величины

live objects
700 MB

free heap
1.6 GB

released heap
900 MB

RSS
3.2 GB

HeapAlloc упал → RSS не обязан упасть синхронно.

Сначала память становится free для Go; потом часть может быть released OS.

GOMEMLIMIT глубже: soft budget и GC CPU limiter

**GOMEMLIMIT
900 MiB**

**live set
850 MiB**

**runway
≈ очень мало**

**GC CPU
limiter**

Runtime может временно превысить soft limit, чтобы не застрять в почти бесконечном GC.

Soft limit защищает progress. Hard cgroup limit всё равно может OOM-kill process.

Go runtime
GOMEMLIMIT = soft



cgroup
memory limit = hard

Нужен headroom на memory вне прямого GC budget: native/cgo, mappings, stacks, platform overhead.

GOMEMLIMIT = container limit — плохой механический default.

Churn и retention — два разных memory-инцидента

Сервис А: churn

```
live heap = 200 MB  
allocation = 8 GB/s
```

```
RSS stable  
GC CPU high
```

alloc_space

Сервис В: retention

```
allocation = 200 MB/s  
live heap:  
500 → 700 → 1000 → 1500 MB
```

inuse_space

Маленький live heap не гарантирует дешёвый GC.

Как internals видны в pprof и runtime/metrics

`runtime.mallocgc`

allocation pressure

`runtime.gcAssistAlloc`

goroutine платит GC debt

`runtime.gcBgMarkWorker`

background marking

`/gc/scan/heap`

scannable heap

`/gc/heap/goal`

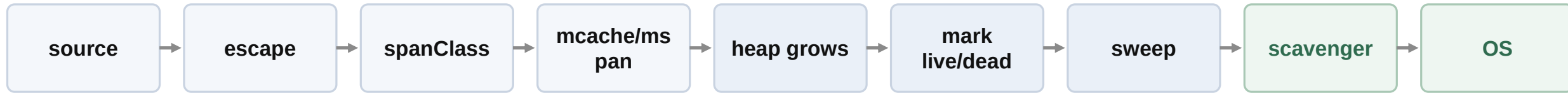
pacer target

`heap/free/released`

allocator vs OS reclaim

Symbol ≠ диагноз. Profile даёт гипотезу, metrics проверяют runtime story.

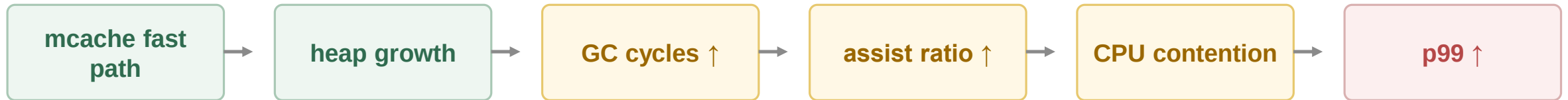
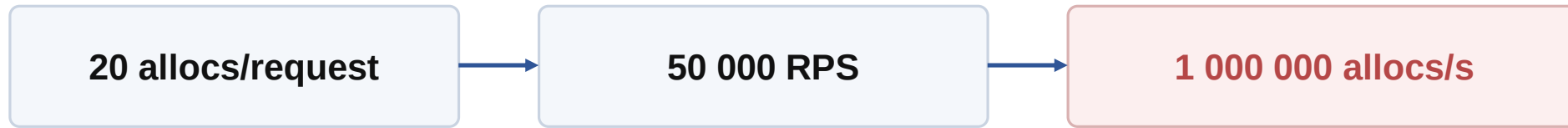
Один heap object: полный lifecycle



Object dead → slot reusable → pages free → physical RSS может уменьшиться позже.

Цена allocation распределена на весь lifecycle, а не только на момент `new`.

Под нагрузкой маленькая allocation превращается в runtime workload НАГРУЗКА



Allocator fast path может оставаться быстрым, а цену service заплатит позже через GC.

`sync.Pool`: снижает churn, но имеет цену

Что выигрываем

- reuse objects/buffers
- allocation rate ↓
- GC work ↓
- меньше temporary garbage

Что платим

- retained memory может ↑
- сложнее lifecycle/correctness
- нужно reset state
- не гарантирует постоянное хранение objects

Pool после profile + benchmark. Не «allocation есть → pool всё».

size classes

scan vs noscan

allocation churn

GOMEMLIMIT

scavenger

pprof + metrics

сделали → нагрузили → увидели runtime symptom → измерили → объяснили
механизм → исправили → снова измерили

- Fast path маленькой allocation — per-P `mcache` + `mspan`; shared allocator уровни нужны для refill.
- `mspan` связывает allocator и GC: slots, allocation state и mark state встречаются в одной сущности.
- Size classes покупают скорость ценой internal fragmentation; scan/noscan влияет на GC cost.
- Green Tea улучшает locality marking work; это текущая реализация, а не гарантия языка.
- Write barrier — цена корректного concurrent graph traversal.
- Heap goal — финишная граница, trigger — старт; pacer подбирает момент по workload.
- Mutator assist напрямую связывает allocation rate с request latency.
- Sweep освобождает memory для Go; scavenger помогает вернуть physical pages OS.
- HeapAlloc, free heap, released heap и RSS — разные уровни.
- GOMEMLIMIT soft; cgroup hard. Слишком tight budget превращает memory problem в CPU problem.

- Go runtime allocator — <https://go.dev/src/runtime/malloc.go>
- mcache — <https://go.dev/src/runtime/mcache.go>
- mcentral — <https://go.dev/src/runtime/mcentral.go>
- mheap / mspan — <https://go.dev/src/runtime/mheap.go>
- GC lifecycle — <https://go.dev/src/runtime/mgc.go>
- GC pacer — <https://go.dev/src/runtime/mgcpacer.go>
- Write barrier — <https://go.dev/src/runtime/mbarrier.go>
- Green Tea — <https://go.dev/blog/greenteagc>
- Go GC Guide — <https://go.dev/doc/gc-guide>
- runtime/metrics — <https://pkg.go.dev/runtime/metrics>

Все конкретные runtime structures и constants помечаем как версионные implementation details.